Augmenting News Articles To Improve Trust

Author:
Ollie Jaffe,
3rd Year of Study

Supervisor:
James Archbold

Abstract

Large language models (LLMs) have excelled at many natural language processing tasks in recent years, producing text that can match the style of humans in certain domains. Malicious actors could use such models for rampant misinformation, impersonation and much more. This problem will only worsen as LLMs become more powerful and accessible. Research in automatically detecting the provenance of some text has been extensively conducted, yet developed classifiers are difficult to use and access. In this paper, we present several accurate classifiers to detect text provenance, all integrated in an accessible application.

Keywords: *Natural Language Processing, Language Modelling, Transformers, Interpretability, Classification, Textual Provenance*

Acknowledgements

I am very thankful to my project supervisor, James Archbold, for guiding me throughout this project and enabling me to pursue my passion. I am also grateful towards friends and family who supported me in writing this.

Contents

Contents 3
1 Introduction 4
2 Objectives 6
2.1 Classifiers 6
2.1.1 Accuracy 6
2.1.2 Scalability 6
2.1.3 Robustness 6
2.1.4 Transparency 6
2.2 Interface 6
2.2.1 Accessibility 6
2.2.2 Explainability 7
2.2.3 Robustness 7
3 Background 7
3.1 Representing Words Computationally 7
3.1.1 One-Hot Encoding 7
3.1.2 Word Embeddings 8
3.2 Language Modelling 9
3.2.1 n-gram Language Models 9
3.2.2 Neural Language Models 10
3.2.3 Recurrent Neural Networks 11
3.2.4 LSTMs, GRUs 12
3.2.5 Transformers 13
3.3 Manual Classification 14
3.4 Automatic Classification 14
3.4.1 Decision Trees 14
3.4.2 Random Forests 14
3.4.3 Bag-of-words 15
3.4.4 Total Probability 15
3.4.5 Detecting Artefacts 16
3.4.6 Transformer 16
3.5 Hybrid Classification 17
4 Classifiers 17
4.1 Decision Trees 18
4.2 Random Forests 18
4.3 Bag-of-Words 19
4.3.1 Single-layer Network 19
4.3.2 Binary BoW 20
4.3.3 Multi-layer Network 21
4.4 Total Probability 21
4.4.1 Experiments 22
4.4.2 Robustness 23
4.5 Transformer 24
5 Adversarial Attacks 24
5.1 Techniques 25
5.1.1 Homoglyphs 25
5.1.2 Misspelling Attack 25
5.2 Results 25
6 Interface 26
6.1 Architecture 26
7 Evaluation 28
7.1 Solution 29
7.2 Interface 29
7.3 Appraisal 30
7.4 Limitations 30
8 Conclusion 31
8.1 Future Work 31
8.1.1 Watermarking 31
8.1.2 Interpretability 31
8.1.3 Auxiliary Signals 32
8.1.4 Robustness 32
8.2 Legal, Ethical and Social Issues 32
8.3 Project Management 32
References 34

1 INTRODUCTION

Modern language models have proven to be surprisingly capable at a number of natural language tasks; even zero-shot evaluation has been shown to perform well. [5, 24]. Scaling these language models and revising the architecture has given us powerful black boxes which produce coherent, semantically and syntactically correct short texts [4, 23]. These LLMs can easily be deployed to solve real-world problems with varying success. They can take natural-language descriptions of a problem as input, produce a wide domain of outputs, and can improve accuracy by being given a few example answers. Many altruistic applications involving such language models have been developed; from finance management to open-ended chatbots to fraud disruption [14].

Looking more into the future, language models may be crucial to solving *AI Alignment* [9]. The transparency of their outputs along with research into mechanistic interpretability have led many working on "prosaic alignment" [1] to believe language models are a basis for a safe and robust aligned AI [2, 20].

Despite the success and potential, LLMs can be dangerous when used by a malicious agent. They can be used to synthesise mass propaganda, fake news articles, impersonate a specific person and much more [32]. Historically, creating these outputs would require a large dataset and training period, thus requiring significant effort and technical ability. However with the recent success of zero- and few-shot prompting in modern LLMs due to their generalizability, altering the output has become trivial by giving a natural language description of the desired output and providing several examples [5]. This approach is instantaneous and requires very little technical ability. Using this tactic, LLMs can quickly be adapted to produce dangerous content. This is partly due to the inherent loss function used by such LLMs; cross-entropy loss over the predicted probability distribution against the target label, given all previous words [22]. If all previous words follow some common theme (e.g. propaganda, sexism, racism &c.), the produced output will follow such theme.

As the capabilities of these LLMs increase, the number of malicious applications will also increase. Recent developments have allowed these LLMs to do more than just synthesise text. For example, OpenAI has researched and implemented the ability for LLMs to edit text, allowing for existing text to easily be revised and rewritten. Malicious agents could use this to rewrite entire news articles to suit their narrative.

Even if there exists a significant difference in quality between machine- and human-written text, LLMs can still be effective. Agents could generate a large number of outputs and select the best, or use the outputs as a basis for their own writing [2].

So far we've considered uses with a malicious agent, however, dangerous outputs can be produced even with good intentions. *Gopher*, a 280 billion parameter model trained by DeepMind [3] in 2021, was trained on *MassiveText*, an English-language text dataset collected from a variety of sources: web pages, books, news articles and code

[1] https://ai-alignment.com/prosaic-ai-control-b959644d79c2

[2] https://www.sudowrite.com/

[3] https://deepmind.com/

[20]. Due to the size of this dataset, it is impractical for humans to manually trawl through it and remove all toxic texts, which inevitably will occur in datasets this large. This gives rise to a small chance of *Gopher* producing toxic outputs and allows *Gopher* to continue toxic text when prompted. Automatic methods exist to prune toxic examples during pre-training, but this is often avoided as separate language model-based classifiers can be trained to recognise toxic outputs. These problems are predominantly being addressed downstream rather than pruning the dataset, with methods such as fine-tuning or monitoring of outputs [33].

Outputs containing false or poor information are likely to be inadvertently produced, which can be dangerous. Rae et al. [24] studied the potential of material harm through such outputs; they discovered that even well finetuned LLMs produced texts that could be dangerous if believed. For example, incorrect advice in medical prescriptions may lead a user to serious harm. Users may place high trust in these LLMs; on the surface, they can appear to be a form of generalised artificial intelligence.

A report published alongside the final release of the GPT-2 models in 2019 found no evidence of direct misuse in public forums and saw that those discussing potential misuse demonstrated limited technical understanding of ML [27]. However the field is rapidly moving towards more accessible LLMs; some can now be accessed through a simple API call [4], and their outputs can be rapidly tuned using few-shot learning. Furthermore, LLMs have overcome the hurdle of simple semantic and syntactic errors, making manually recognising outputs difficult [5].

Using humans to distinguish has been extensively researched, with most participants in experiments achieving just above random accuracy when evaluating short texts from GPT-3 in specific domains [5, 12]. This results from two important factors; *exposure bias* and human evaluation techniques. Exposure Bias, which we will discuss more later, is a problem in training LLMs leading to increasingly different perplexity between generated- and human-written text, thus shorter generated texts will prove harder to classify [25]. Humans typically approach the task of distinguishing by evaluating the text's meaning and coherency in linking ideas, which LLMs can perform well for short texts on simple tasks (e.g. generating news articles). Evaluation of "lower-level" details of short texts is required to accurately distinguish, which humans struggle with [16].

Significant research has been conducted in creating accurate classifiers using artificial intelligence [17], yet we could not find a publiclly-accessible application that integrates one of these tools. The difficulty of the task of distinguishing provenance between human and machine, along with the rate at which these LLMs can produce content, motivated the requirement for a tool to accurately identify LLM outputs to counter malicious actors and dangerous outputs. We propose our application, a proof of concept browser extension that classifies whether some text was produced by a human or LLM.

[4]https://openai.com/api/

2 OBJECTIVES

To ensure we made a significant contribution to the field, we devised several objectives for our application. Our application was designed to be a client-server with a frontend, and backend that hosted the classifiers. We split our objectives into two, depending on whether they related to the classifiers or the interface.

2.1 Classifiers

Our solution relies on classifiers to decide the provenance of the text. We decided on several high-level requirements that these should fulfil to guarantee the effectiveness of our solution.

2.1.1 Accuracy. Our best classifier must achieve high accuracy of above 90% when tested on classifying human text, and generations from a specific language model. Additionally, we should predict false negatives more than false positives; incorrectly labelling some human text as human-written is not desired.

2.1.2 Scalability. Our classifiers' must be scalable and computationally efficient to run. It is possible to generate thousands of outputs per hour from an LLM with a modern high-end computer, which is problematic when a malicious agent exploits this. This quality would improve the applicability of our model, for example, we could integrate it into social media platforms to automatically flag posts.

2.1.3 Robustness. Due to the rapidly-moving state of modern natural language processing (NLP), we must construct our classifiers' such that they retain high accuracy when tested on a diverse range of architectures. State-of-the-art (SOTA) language model parameters have been increasing at an exponential rate since 2018 [3]. Model configurations also vary, whether it's generation techniques, loss functions or sampling. Different tasks require different configurations, thus they commonly change. Furthermore, adversarial attacks have been shown to neuter many classifiers. Simple black box attacks, such as replacing characters with homoglyphs, have taken classifiers from almost-perfect accuracy to worse than random [35]. We must ensure that developed classifiers are robust to all of the above.

2.1.4 Transparency. Explaining why modern AIs make decisions is one of the hardest problems in the field. We often use massive architectures with billions of parameters which are effectively massive black boxes. However, we will try our best to offer limited interpretability of our classifiers' reasoning to users so they can come to understand specific decisions and have better judgment on whether to trust the classifiers or not.

2.2 Interface

Our classifiers' are accessible by users through an interface. To ensure the best user experience, and to enable the full ability of our classifiers', we created some objectives to follow.

2.2.1 Accessibility. The weights and architectures for many accurate classifiers have been released, but they have never been integrated into an accessible interface. Most do not have the technical ability to use these classifiers themselves. Additionally, even the most efficient modern classifiers

would struggle to run on the average consumer computer [17]. Our interface must seamlessly integrate with web browsers since most digital information is published on the internet. Furthermore, it must be intuitive and simple to use; we do not expect users to have an understanding of the underlying machine learning to be able to use this. Explanations should be offered where appropriate to guide the user and explain how our interface works. Finally, the interface should not host the classifiers directly. They should be hosted on a separate computer, which can be communicated with to perform inference on given inputs. Making pre-trained classifiers accessible is critical for democratising this technology.

2.2.2 Explainability. The methodology of each classifier should be adequately explained so users understand how each of the classifiers work, and can make an informed decision on which ones to use. The raw outputs of each classifier should be shown to the user so they have the option to see how the models individually performed. Interpretability of the models should be appropriately visualised to the user if implemented. Finally, the exact methodology of each classifier and the methodology for calculating the final output should be publiclly accessible.

2.2.3 Robustness. Since our interface will be accessible across the internet we expect it will take many different inputs. Our classifiers' take plaintext as input, thus we need to ensure any input texts are adequately cleaned and converted to the correct encoding. Furthermore, our interface should gracefully react when our classifiers' return an error, for example, if the given input is too long. Additionally, our interface should readily report any user-created errors to communicate their mistakes so they can correctly use the interface.

3 BACKGROUND

In this section, we provide background for the theory and technologies we used when developing this application. This is a result of a broad literature review we conducted to survey the state of the field and understand employed techniques.

We'll first discuss the fundamental task of language modelling, and analyse several architectures that solve the task. For each architecture we'll discuss strengths and flaws, so we can decide which are relevant to our problem. Next, we'll look at different methods of classifying text between human- and machine-written: manual, automatic, and hybrid methods.

3.1 Representing Words Computationally

Most machine learning architectures can't parse the raw input texts directly; they require such texts to be represented in some numeric state. There are many different methodologies for solving this problem, but in this section, we'll discuss two that were relevant in our solution.

3.1.1 One-Hot Encoding. For every word $m_i \in V$, where V is our vocabulary containing every word in our language, we construct a vector $h_i \in \mathbb{R}^{1\times|V|}$. Each vector contains a single "1" value in a unique position, all other elements are "0". This gives each

word a unique vector representation which can be processed by machine learning models. Below is an example of this encoding method:

$$\begin{aligned} \text{dog} &\rightarrow [1, 0, 0] \\ \text{cat} &\rightarrow [0, 1, 0] \\ \text{parrot} &\rightarrow [0, 1, 1] \end{aligned} \tag{1}$$

This solution is far from perfect. All vectors are orthogonal to one another, thus there is no similarity between any. Ideally, we'd want to encapsulate this quality in our vectors, such that similar words will have similar vectors.

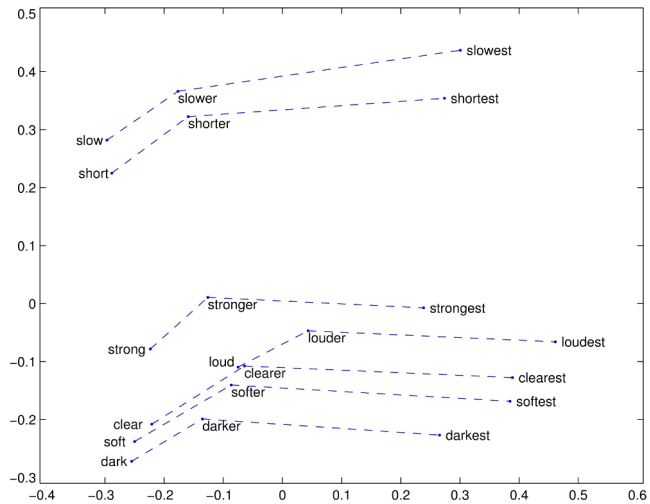

Figure 1: GloVe embeddings visualised, with word relationships highlighted

3.1.2 Word Embeddings. These are a way to computationally represent words whilst inherently capturing their meaning and relationship to other words through their value.

In our methodology, each word is represented by a dense vector with the aim for similar words to be close to one another in Euclidean space. Furthermore, the vector between different words should represent their relationship. An example of these relationships is visualised in Figure 1; we can see that the vector from the embedding of "strong" to "stronger" is similar to that from "slow" to "slower". Word embeddings offer an advantage over one-hot encoding; they allow for the meaning of words to be inputted into the network, rather than the network having to learn such meanings itself.

To create these embeddings, we first perform a process known as *tokenization.* We construct a bijective mapping between words and a subset of the natural numbers, such that every word has a unique numerical representation, to which we refer as its token. Using this technique we can represent a text as a sequence of numbers. Note, methods of tokenization, such as Byte Pair Encoding [10], also map individual characters and sub-words to unique numbers.

We then map a set of tokens onto embeddings. We will be using a pre-trained set of word embeddings since many exist and it can be computationally expensive to train them. Pennington et al. [21] proposes an unsupervised learning algorithm for obtaining word embeddings, which results in SOTA embeddings that generalise to a wide domain of tasks. Many modern architectures incorporate the task of learning word embeddings in their solution as an instrumental task, which can result in high-quality embeddings [5, 23]. This will require a large dataset of texts, which is commonly referred to as a *corpus.*

3.2 Language Modelling

Language modelling is the task of predicting what word comes next given a previous sequence of words. Given some sequence of words $x_1, x_2, ..., x_n$ you compute the probability distribution of the word x_{t+1}, where x_{t+1} can be any word in the vocabulary $V = \{m_1, m_2, ..., m_{|V|}\}$:

$$P(x_{t+1}|x_1, x_2, ..., x_t) \tag{2}$$

This method can be iteratively applied to generate arbitrarily large texts. Using the predicted probability distribution $P(x_{t+1}|x_1, x_2, ..., x_t)$ one could use some sampling method to choose some word from such distribution, such as selecting the word with the highest probability. Next, append x_{t+1} to the previous sequence of words, and predict the word x_{t+2} using such sequence: $P(x_{t+2}|x_1, x_2, ..., x_t, x_{t+1})$. We can repeat this as many times as necessary.

3.2.1 n-gram Language Models. An n-gram model counts the frequency of every permutation of n words from our vocabulary V within some corpus. Using such frequencies, we can calculate probabilities of sequences of words occurring, thus we can solve the language modelling problem. This relies on the Markov Assumption; x_{t+1} depends only on the preceding $n-1$ words, thus:

$$P(x_{t+1}|x_1, x_2, ..., x_t) = P(x_{t+1}|x_{t-n+2}, ..., x_t) \tag{3}$$

Under this assumption we can rewrite equation 2:

$$\begin{aligned} P(x_{t+1}|x_1, x_2, ..., x_t) &= \frac{P(x_{t-n+2}, ..., x_t, x_{t+1})}{P(x_{t-n+2}, ..., x_t)} \\ &\approx \frac{\text{freq}(x_{t-n+2}, ..., x_t, x_{t+1})}{\text{freq}(x_{t-n+2}, ..., x_t)} \end{aligned} \tag{4}$$

We can solve equation 4 using the stored phrase frequency distribution by simply dividing the number of occurrences of $(x_{t-n+2}, ..., x_t, x_{t+1})$ over the number of occurrences of $(x_{t-n+2}, ..., x_t)$ within our corpus. This is computationally inexpensive when used for testing, however, training such a model is expensive due to a large number of phrases with a very low number of occurrences. A technique known as *filtering* can be applied to remove such phrases, which reduces the cost.

N-grams can be applied for language modelling to generate text. To obtain a probability distribution of the next word x_{t+1} over the vocabulary V given context $\{x_1, x_2, ..., x_t\}$, for each $y \in V$, let $x_{t+1} = y$ and use equation 4 to give us a probability. Concatenating all these probabilities will result in our distribution. With small n this often leads to poor quality outputs that are likely to be semantically or syntactically incorrect, as the model can only factor minimal context into its prediction. Increasing n alleviates such a problem but incurs a significant computational and storage cost as the model stores counts for all n-grams in the corpus.

This approach also has an inherent problem known as the *sparsity problem*. If some phrase x_{t-1}, x_t, x_{t+1} never occurs in the data then a bigram (2-gram) using equation 4 will give it a probability of zero, and thus will never be predicted. A solution, known as *smoothing*, is to add a small δ to every count $m \in V$. Furthermore, if $\{x_{t-1}, x_t\}$ never occurs then

our model will give a probability of zero for every predicted word $m \in V$, making it impossible to predict the next word. One solution is to condition on just x_t, "downgrading" the bigram to a unigram, which is known as *backoff*. This approach works well as smaller phrases are more likely to be present in some corpus. Increasing n makes this sparsity problem worse, as it decreases the probability of a phrase of n words appearing in the corpus.

Due to both discussed constraints, typically $n = 4$ is used. However, knowing of the previous three words is insufficient for producing accurate text. Errors are common, and this approach struggles to link together ideas over multiple sentences.

3.2.2 Neural Language Models. This approach takes inspiration from artificial neural networks.

Given context $\{x_1, x_2, ..., x_t\}$ and window size n, we discard all words that lie outside our "window" to give $\{x_{t-n+1}, ..., x_t\}$ as our context. We represent our new context using one-hot encoding, then get respective word embeddings and concatenate them to get $e = [e_{t-n+1}; ...; e_t]$. These are then passed through a linear layer and some non-linearity f:

$$h = f(We + b_1) \tag{5}$$

Our intermediary h is then passed through another linear layer and softmax function to get a probability distribution over all $w \in V$:

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|} \tag{6}$$

Training this model for our task requires a large corpus of text. For a window size of n we get every sequence of n consecutive words in such corpus, and input each sequence into our model. Our loss function is the cross-entropy loss between our output probability distribution $\hat{y}$ and the true next word y. Since there's only one correct label, we can say:

$$y_i = \begin{cases} 0 & \text{if } w_i \neq y \\ 1 & \text{if } w_i = y \end{cases} \tag{7}$$

Thus we can simplify the cross-entropy loss calculation, as seen below:

$$\text{CE}(y, \hat{y}) = -\sum_{w \in V} y_w \log \hat{y} = -\log \hat{y}_{x_{t+1}} \tag{8}$$

This is somewhat similar to the n-gram approach, since we rely on a Markov Assumption to take a subset of the previous words, however, the linear layer combined with the non-linearly allows a more complex function to be modeled rather than just a simple frequency count. Furthermore, the use of word embeddings codes some inherent meaning of each word in the inputs, which interact with each other.

This method has a few significant improvements over the previous n-gram approach. Firstly, there is no need to store all n-gram counts, thus the storage requirement is significantly lower. Secondly, this approach doesn't suffer from the *sparsity problem* so any sequence can be inputted into the model without alterations.

However, similar to n-grams, this approach is limited by its window size. Enlarging the window also enlarges the weight matrix W. Ideally, we'd set out window size to be equal to the full length of the contextual sequence, $n = t$, but this can be computationally infeasible. Additionally, this wouldn't guarantee that the whole input is considered, as we may have some input sequence $Z = \{z_1, ..., z_p\}$, with $p > n$, which is longer than our window.

Furthermore, all input word embeddings are multiplied by different weights in W, so each set of weights that is multiplied by an embedding will have to individually learn the same task. This isn't ideal - there should be symmetry in how inputs are processed, and some learned tasks should be shared amongst weights.

3.2.3 Recurrent Neural Networks. A Recurrent Neural Network (RNN) model allows variable-length inputs to be processed across multiple time steps, unlike traditional feed-forward neural networks which take all inputs at once and inherently encodes linear locality between inputs [26].

Given a sequence of inputs $x_1, ..., x_t$ and an initial hidden state h_0, at every timestep $i \in [1, t]$ we compute the new hidden state using some formula, for example:

$$h_i = \sigma(W_h h_{i-1} + W_x x_i + b_1) \tag{9}$$

where W_h is the weight matrix mapping hidden states to hidden states, W_h is the weight matrix mapping inputs to hidden states, and σ is some non-linearity.

We can use RNNs for our language modelling task. Given a sequence of inputs $\{x_1, ..., x_t\}$ we get the one-hot encodings, then retrieve and concatenate the associated word embeddings to get $e = [e_1; ...; e_t]$. For our initial hidden state h_0, we could either set this to the zero-vector or allow it to be learnt by the model.

At every timestep i, we calculate the new hidden state h_i using equation 9 with our embedding e_i as input x_i. We follow this until we've calculated the final hidden state h_t. This hidden state vector should represent the meaning of the whole input passage. We pass this through a linear layer with softmax to get a probability distribution over our vocabulary V:

$$\hat{y} = \text{softmax}(Uh_t + b_2) \in \mathbb{R}^{|V|} \tag{10}$$

For training our model, we could feed our entire corpus into the RNN and compute the output distribution at every timestep i. However, computing the loss and gradient for the entire corpus is too computationally expensive, so we split our corpus into sentences of length T and consider each sentence as a separate input to our model. Our loss function is cross-entropy between each predicted probability distribution $\hat{y}$ and the actual next word y, as shown in equation 8.

We can augment our model to continuously generate text; at every timestep $i > g$, where g is some index signalling when to begin generation, we compute a probability distribution over our vocabulary V. We select a word $w \in V$ using this distribution with a sampling technique and input this into the RNN in the next timestep. We can quickly generate additional words rather than having to pass the entire input through the model, as we had to for previous architectures.

This approach has several advantages over the previously discussed ones. As we mentioned before all inputs inherently encode linear locality due to how inputs are processed over time; knowing the order of words is critical for accurately predicting text. Our neural language model approach didn't encode the ordering in its inputs which gives rise to ambiguity on what the true ordering is. In theory, due to the presence of the hidden states, when computing new hidden states we access information from many previous hidden states.

Since the same weights are applied at every timestep our model can process any length input, unlike the two previous approaches which couldn't change without retraining the model. This also results in the model size being the same for all length inputs, unlike n-grams. Finally, since the same weights are used on every timestep; there is symmetry in how the inputs are processed, which the neural language model lacked.

Unfortunately, two inputs take $O(s)$ to interact, where s, the *Sequence Length*, is the number of RNN units between them. Large sequence lengths result in semantic and syntactic information being lost, leading to inaccurate outputs. The linear order of wording being encoded in the inputs can be helpful but isn't the right application for every task; there are many different types of positional encodings [11].

Another problem to consider is the *vanishing gradient problem*. If we apply the chain rule to equation 8, we get:

$$\frac{\delta h_t}{\delta h_{t-1}} = \mathrm{diag}(\sigma'(W_h h_{t-1} + W_x x_t + b_1)) \tag{11}$$

If we look at the gradient of the loss on timestep i, $J_i(\theta)$, with respect to the hidden state from a previous timestep j we get:

$$\begin{aligned} \frac{\delta J_i(\theta)}{\delta h_j} &= \frac{\delta J_i(\theta)}{\delta h_i} \prod_{j<t\leq i} \frac{\delta h_t}{\delta h_{t-1}} \\ &= \frac{\delta J_i(\theta)}{\delta h_i} W_h^{i-j} \prod_{j<t\leq i} \frac{\delta h_t}{\delta h_{t-1}} \end{aligned} \tag{12}$$

If the weight associated with the hidden state W_h is small, then the gradient will be increasingly smaller as i and j become further apart. This results in gradient signals not travelling far during backpropagation, thus model weights are only updated from nearby computations; this is similar to having a fixed small window. The vanishing gradient problem is present in all neural network architectures, however, RNNs are particularly vulnerable due to repeated multiplication using the same weight matrix.

A potential solution is to add more direct connections in our network, which allows the gradient to flow further during backpropagation [13].

One final consideration is that we can't compute hidden states in parallel; future hidden states can't be computed until all previous states are.

3.2.4 LSTMs, GRUs. Whilst vanilla RNNs can store information between states, the hidden state is continuously being rewritten, making it difficult to preserve information and recognise long-term dependencies. Long short-term memory (LSTM) is a slightly altered version of vanilla RNNs where each unit has three gates: the input gate, output gate and forget gate [15]. These control the storage and flow of information through each LSTM unit.

By simply setting the forget gate to remember everything, in theory, we can preserve all information in each unit indefinitely. The values associated with each gate are learned during the training process, thus an LSTM will learn what information is useful for giving the correct output. LSTMs provide a partial solution to the vanishing gradient problem, as they preserve information across multiple units better than RNNs.

Cho et al. [7] proposes a simpler alternative to LSTMs, named gated recurrent unit (GRU). This approach attempts to mirror the capabilities of LSTMs whilst removing unnecessary complexities.

GRUs are quicker to compute than LSTMs and have few parameters, but are worse at conserving long-term dependencies.

Despite the improvements over vanilla RNNs, both LSTMs and GRUs still have a significant flaw; only one unit can be computed at a time.

3.2.5 Transformers. This architecture is proposed by Vaswani et al. [30], and has been the basis for many architectures across many different domains of tasks. These typically consist of an encoder and a decoder block however Liu et al. [18] found that by stacking "Transformer-Decoder blocks", which are based on the original decoder, they are capable of language modelling. These transformer-decoders fundamentally have two layers: self-attention and position-wise feedforward.

Self-attention layers have their keys, queries and values all come from the input vector. Each has a unique learnable matrix, K, Q, V respectively, to allow different aspects of the input to be utilised in each job. For some input vector $\{x_1, ..., x_t\}$, with $x_i \in \mathbb{R}^d$, we first concatenate the inputs:

$$X = [x_1; ...; x_t] \in \mathbb{R}^{t \times d} \tag{13}$$

Then we compute all the pairs of attention scores:

$$(XQ)(K^T X^T) = XQK^T X^T \in \mathbb{R}^{t \times t} \tag{14}$$

Finally we normalise the scores and multiply the scores with the values to get our output:

$$\text{softmax}(XQK^T X^T)(XV) = Z \in \mathbb{R}^{t \times d} \tag{15}$$

Self-attention allows a layer to compare every input against every other, which cultivates long-term dependencies. Furthermore, the output will differ from the input depending on the attention scores, allowing our network to construct complex representations factoring in all inputs. Our network also uses position-wise feedforward layers; these are a type of feedforward layer which is applied to every input with the same weights for each layer.

These two types of layers are typically stacked on top of each other many times. Transformer-decoder networks have been proven to be capable at a wide range of tasks and are extremely versatile; the hidden states of the last layer can be taken and applied to a wide domain of tasks, such as language modelling. For example, we could take the last hidden state and input it into a simple feedforward neural network to predict the token with the highest probability.

Transformers solve many problems we highlighted with other architectures. Long-term dependencies are feasible in this architecture, due to repeated multi-headed self-attention. They don't suffer from any vanishing or exploding gradient problems, and allow for each of their inputs to have a continuously-updating representation. Furthermore, the computation grows linearly with respect to the input dimensionality d, allowing for very high-dimensional representations of tokens.

However, with very large inputs transformers can be intractable. Computing self-attention results in our computation growing quadratically with the length of the input, whereas RNNs grow linearly.

3.3 Manual Classification

As we noted earlier, human classification has been extensively researched; typically a study judging human capability is performed alongside the release of a new LLM. Almost all studies performed on language models in size equal to or greater than GPT-2 have shown that humans correctly classify 50±5% of the time [5, 12, 23]. However, most of these studies are flawed. They often use datasets of texts from a simple domain that contain statements which are difficult to verify. A popular option is to use news articles; these rarely contain high-level technical analysis, and test participants will struggle to verify the claims of some arbitrary news article. A better measurement would be comparing examples from many different domains: general knowledge, maths, coding and more. Due to this, human classification accuracy is significantly higher than reported when tested in an open-ended environment.

Despite this, manual classification alone is not good enough. A single malicious actor could automate thousands of social media accounts to post machine-generated tweets; manual classification won't be able to scale to meet this demand. As the capabilities of LLMs increase humans will get increasingly worse at classification. Automatic inexpensive classifiers are required to combat this problem.

3.4 Automatic Classification

In this section we analyse fully automatic classifiers that require no intervention. Even the simplest of approaches achieve significantly higher than random accuracy, which suggests that the probability distribution of predicted words from LLMs is very different to that of human text.

3.4.1 Decision Trees. Given some text T, we first tokenize T to get K, then get and concatenate the corresponding word embeddings e_i for every token $k_i \in K$, giving us vector $E = [e_1; ...e_{|K|}]$. Using these word embeddings we calculate a single vector that represents all of them; this vector should represent the meaning of the input text T. Many different methods exist, the simplest is to take the average of all embeddings:

$$A = \frac{1}{|K|} \sum_{i=0}^{|K|} k_i \tag{16}$$

We then fit a decision tree using vector A. Intuitively, this should find what ranges in specific features are indicative of the text being human- or machine-written.

This is our simplest classifier, whose accuracy depends on the encoding process. Simple, low-dimensional encodings will fail to encapsulate enough information to make an accurate classification. Additionally, deep decision trees are prone to overfitting. Providing a large training dataset or limiting the depth of the tree will avoid this.

Training this model is quick and simple. All we require is a dataset of example outputs with labels; no access to the original generating language model is required.

3.4.2 Random Forests. These are an ensemble of multiple decision trees, where the final classification is given by the most commonly predicted class from all the trees. *Bagging* is employed, where each tree randomly samples from the dataset with replacement. Additionally, when splitting a node

only a random subset of features is considered. This results in trees that are trained using different data, which use different features to classify. Random Forests are renowned for being a great machine learning algorithm since they're incredibly accurate and robust to overfitting. Unfortunately, they do not possess the interpretability of decision trees.

We could apply a similar pipeline to our Decision Trees; tokenize and embed the input texts, calculate the average embeddings then use these to fit the model. Once again, accuracy for this task will depend entirely on the quality of our word embeddings.

3.4.3 Bag-of-words. Given some text T and vocabulary V, for every $m \in V$ we count the frequency of such token within the text to give us f_i. We then construct frequency vector $F = [f_1; ...; f_{|V|}] \in \mathbb{R}^{|\mathbb{V}|}$. This vector is then fed into a logistic regression model to predict whether the text was human- or machine-written.

We expect that all LLMs produce certain words significantly more or less frequently than the average human text, since they do not have human-level intelligence. Specific sampling techniques which we will discuss later, such as Top-K or Top-p, also further diverge the output. A simple classifier that would identify such a pattern would be a 1-layer feed-forward neural network:

$$\mathrm{o} = \mathrm{sigmoid}(FW + b) \tag{17}$$

It may also be the case that combinations of words appear more or less frequently; our previous architecture would struggle to identify this pattern due to only having a single layer. Increasing the number of layers and adding non-linearities would make our model recognise more complex patterns such as this.

Unfortunately, this model does away with the spatial locality of words, thus completely losing the meaning of large texts. However, it is a simple, small model that is quick to train and deploy.

3.4.4 Total Probability. A simple yet surprisingly accurate zero-shot detection method thresholds the probability of some text to classify. We expect the probability of generating human text to differ from that of machine text. Given some discriminator LLM, we define the total probability of a text as the sum of the log-probabilities of each token being produced given its context; we can imagine this total probability as a rough estimate of such LLM producing the text. More formally, given some discriminator D and text, $T = \{t_1, t_2, ..., t_n\}$, our text probability is:

$$\begin{aligned} P(T) &= \prod_{1}^{n} P(t_i|D, t_1, ..., t_{i-1}) \\ &= \sum_{1}^{n} \log(P(t_i|D, t_1, ..., t_{i-1})) \end{aligned} \tag{18}$$

Note, the text T is produced by some language model generator G, which isn't always the same as D. Ideally, discriminators would be robust enough to correctly classify text from a diverse range of generators.

This method is proven to achieve a large accuracy; OpenAI [5] tested this on their largest GPT-2

[5] https://openai.com/

model and achieved an accuracy of 76% [27]. Furthermore, this approach has been shown to be robust when the model for the text generator differs from the discriminator, however altering the sampling technique significantly alters the performance of this method.

This approach requires access to the output probability distribution over the vocabulary, or the activations of the last layer of the language model head. This will require full access to the language model. However, in the case where we only have access to the word with the highest probability, we can approximate the probability distribution by repeating the language modelling process for a static input n times and counting the frequencies of the outputted words, where n is a large number.

3.4.5 Detecting Artefacts. Outputs from LLMs leave "artefacts" in their text; these vary depending on the specific model that was used and the machine configuration. These artefacts are often subtle and difficult to manually spot, for example predicting a specific phrase with a slightly higher frequency.

Tay et al. [28] proposes a method to detect these artefacts and map these to a specific generating language model and configuration. They developed five models, from LSTMs to convolutional neural networks to a transformer-encoder. Most took the set of input embeddings of the original text as input. Their simplest model was a Linear BoW model that averaged the word embeddings and passed them into a single-layer linear classifier. This identified the correct machine configuration with an accuracy of 47.86%, above the random chance of 29.33%. Their most complex model, the transformer-encoder, only achieved an accuracy of 48.74%. They found that this approach best identified different configurations, but struggled to identify the correct underlying language model.

They adapted this approach by adding human-generated articles, and they found that the model separated the human-written articles from machine-written at above 90% accuracy. However, this does not represent the true accuracy of their classifier. Their datasets used for training the text-generating language model and for testing the discrimination accuracy are from significantly different domains of news articles, thus their model was a stylistic and topic discriminator rather than performing the true task.

Despite using a diverse range of architectures, they all achieved an accuracy ±1.5% from one another. This suggests that word order isn't important in discrimination, showing artefacts are mostly specific words rather than compositional dependencies (order of words).

3.4.6 Transformer. The current SOTA is to fine-tune an LLM to detect outputs from other LLMs. We first modify the architecture of a transformer-based LLM by removing the language modelling head and then add a logistic classifier in its place. This logistic classifier takes as input the last hidden state of the model and uses this to classify the text. During training the parameters of the LLM-base are frozen, and only the logistic classifier is trained.

Transformer-decoder LLMs build a representation of their inputs in their last hidden state, as this is given as input to their language modelling head. Thus, we can use this representation in a separate

task and allow a separate model to recognise structure in these high-dimensional vectors. Due to the high-dimensionality of the hidden states, this can be a costly model to train.

The RoBERTa language model was fine-tuned to classify outputs from GPT-2 and achieved an accuracy of 95% [27]. Furthermore, it still achieved a high accuracy when tested on smaller GPT-2 models, demonstrating its robustness. Unfortunately, it did not retain accuracy when compared against larger models.

This approach requires almost-full access to the underlying LLM, which is rare in the current climate. Since the release of GPT-3 most SOTA LLMs have been inaccessible [24], or only accessible through a limited API [6]. This significantly restricts the effectiveness of this methodology, as without access to the largest model it will always be ineffective.

3.5 Hybrid Classification

Limited research has also been performed in hybrid classification, where humans are given access to tools to help make their decision. One approach taken followed from the Total Probability methodology discussed in Section 3.4.4, but rather than calculating the total probability of the whole input text, they visualised the probability of every individual token within the text to the user [12]. An example of this tool is shown in Figure 2.

The authors found an average accuracy improvement of 18.1% when using the tool, and a significant amount of users reported it helped them identify fakes faster. This approach has the same flaw as the Total Probability method, as we require full access to the generating language model. Additionally, it requires human labour, which as we noted in Section 3.3 struggles to scale with the potential outputs of language models.

[6] https://openai.com/blog/openai-api/

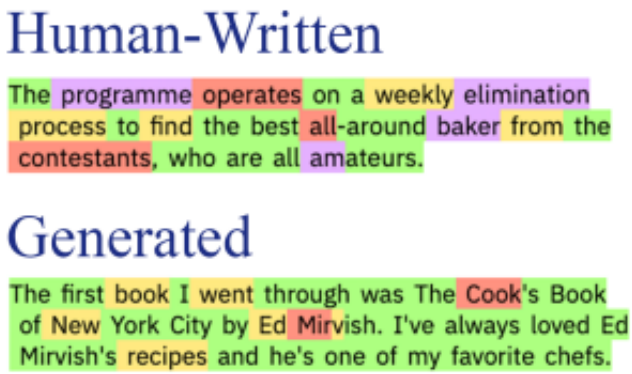

Figure 2: Overlay from GLTR on generated and human-written text

4 CLASSIFIERS

In this section we'll discuss the implementation and evaluation of all classifiers we included in our solution.

For classifiers requiring the use of an LLM, we used GPT-2 [23], a language model trained by OpenAI in 2019. It wasn't feasibly for us to use more powerful models, like GPT-3, for inference and fine-tuning due to hardware constraints. GPT-2 has several publiclly available variations with different numbers of parameters: 117M, 345M, 774M and 1558M. This is useful as we can test how robust our classifiers' are against larger or smaller LLMs.

Additionally, we required a dataset of texts produced by humans and GPT-2 to train our models using supervised learning; ideally within a similar style and domain (e.g. social media post, academic

paper) to prevent any of our models from exploiting that relationship and becoming a stylistic/topic classifier. We used a dataset provided by OpenAI, a collection of 250K documents from the WebText test set, with 250K samples generated by each GPT-2 model fine-tuned on the WebText test set [7].

Most of our models are trained and evaluated on Google Colab [8] due to its easy accessibility and available hardware. Models are implemented using PyTorch [9] because of developer experience. All pre-trained language models are implemented using Hugging Face Transformers Python library [10]; it offers an ever-expanding range of cutting-edge models and allows flexibility in how they are used [34].

4.1 Decision Trees

To test how applying simple models to a historically complex problem performs, we implemented the Decision Tree classifier as a baseline for our other models.

We used the tokenizer trained with GPT-2, which has 768-dimensional vectors representing every token. We then calculated the text representation using an average of all token embeddings. This methodology was applied to every model in the GPT-2 family: 117M, 345M, 774M and 1558M. We got accuracies of 57.96%, 56.55%, 57.83% and 57.04% from the smallest GPT-2 model to the largest respectively, with an average accuracy of 57.35%. Whilst all accuracies were within ±1% of one another, demonstrating significant robustness, all had low accuracies just above random chance.

To experiment with how changing the embeddings alters our accuracy, we also used the tokenizers trained with GPT-1 and GPT-J [22, 31] on GPT-2 117M giving us average accuracies of 56.15% and 57.08% respectively. Only very minor changes were made, which could just be noise. After manually inspecting the top nodes of the decision tree, we saw that all fit decision trees split the same features first.

Decision trees are renowned for being interpretable since they're essentially an if-else statement tree comparing a single feature at a time. However, here we are evaluating hundreds of features that each represent some unknown quality of the text, thus interpretability on this model is difficult.

4.2 Random Forests

We next experimented with Random Forests, as we expected them to give a significant improvement over our Decision Tree.

We used the same methodology as with our Decision Tree: taking the average of token embeddings, a GPT-2 tokenizer and testing on the family of GPT-2 models. After fitting, we got accuracies of 63.68%, 59.52%, 66.96% and 67.43% from the smallest GPT-2 model to the largest respectively. Interestingly, we see that as the size of the generating model increases the accuracy also increases. Perhaps this is due to the quality of the outputs; as the model becomes more complex its outputs become more coherent, reducing the influence of errors.

[7] https://github.com/openai/gpt-2-output-dataset

[8] https://research.google.com/colaboratory/faq.html

[9] https://pytorch.org/

[10] https://huggingface.co/transformers

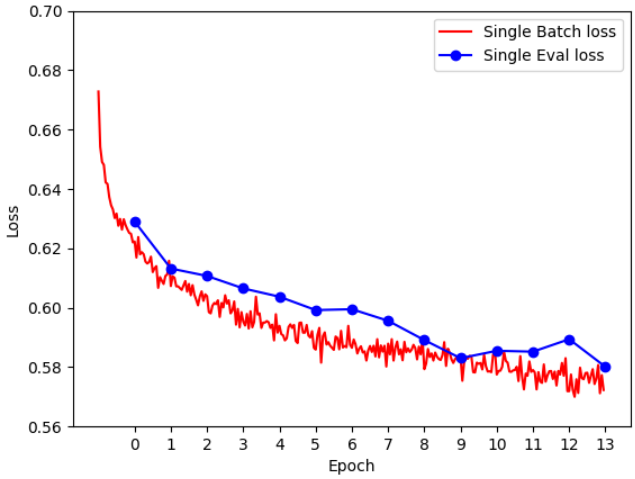

Figure 3: Loss over time for 1-layer BoW neural network

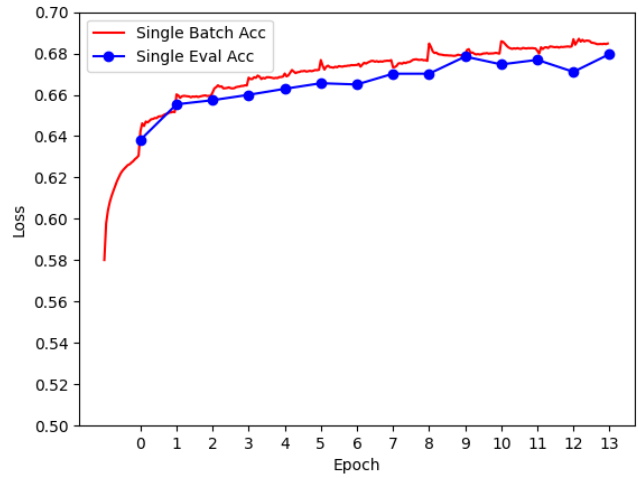

Figure 4: Accuracy over time for 1-layer BoW neural network

4.3 Bag-of-Words

We implemented several variants of the Bag-of-Words (BoW) model and performed extensive interpretability on all where possible.

4.3.1 Single-layer Network. Our initial architecture was a one-layer neural network with binary cross-entropy loss. This was trained on the entirety of the WebText test set over 14 epochs, giving us an accuracy of 67.97%. Figure 3 displays the loss over time, of both the average batch loss and average loss on the evaluation set after every epoch. We can see that the model likely wasn't overfitting as our batch loss on the training set was similar to the loss on the evaluation set throughout. Figure 4 shows the accuracy of this model on each batch of the training set and the accuracy on the evaluation set. We can see that the model rapidly increases its accuracy then stagnates. With more training, we may get up to 70% without overfitting with this architecture.

We hypothesised that this simple method achieved a fairly high accuracy due to the frequency disparity of uncommon words between GPT-2 and human text. During GPT-2's prediction process, it assigns a non-zero probability to every word in its vocabulary. It then samples from this distribution to select which word will be predicted to come next. A lot of words will be syntactically or semantically incorrect so GPT-2 will assign them a very low probability to avoid them being predicted. However, since there exists a significant amount of words like this, cumulatively they will represent a significant portion of the total probability mass. Even a probability as low as 0.1% of predicting one of such words is undesirable, as this would likely happen when generating long texts. Furthermore, due to the inherent task of language modelling we introduced in equation 2, errors will rapidly accumulate as soon as one error is made. To prevent this, two techniques are widely used to cull the tail of the probability distribution. In *Top-K* sampling,

Token	Weight	Token	Weight
including	0.141	Job	-4.78e-08
„	-0.116	smokes	3.72e-07
etc	-0.110	Question	-5.38e-07
..	-0.107	BC	-6.24e-07
ly	-0.099	LA	-9.44e-07

Figure 5: Largest and smallest weights, with respective tokens, of single-layer neural network

we take the top K words in our original distribution into our new set. In *Top-p*, we take the words with the highest probability into our new set until the cumulative probability of such set exceeds p. After getting a new set with either method, we normalise the probabilities to sum to 1 and then sample from this new distribution.

These techniques limit the number of incorrect tokens being produced, but also have the side effect of pruning semantically and syntactically correct words that are used uncommonly. Thus, our BoW model could just be classifying based on the frequency of these uncommon words.

Since we used a 1-layer neural network, the magnitude of the weights out each input neuron i correlates to the amount token i impacts the classification. Thus, to test our hypothesis we took the weights from our best-trained BoW model and found which tokens were associated with the largest weights. Figure 5 displays the tokens with the largest and smallest weights. We see that some uncommon tokens "„" and ".." have large weights, but "including" had the largest weight and isn't an uncommon word.

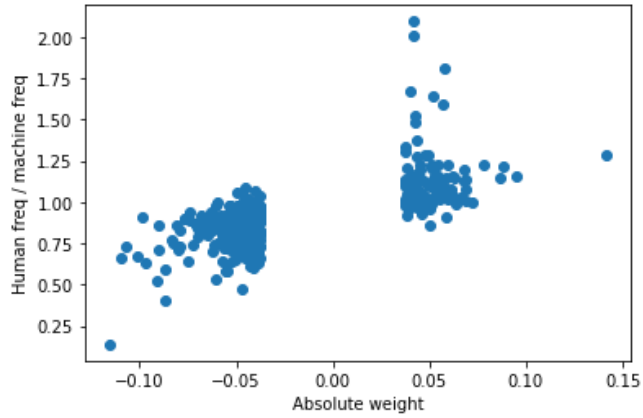

Figure 6: Largest 500 weights plotted against the proportion of occurrences belonging to labels

The presence of fairly common words led us to believe that another factor may be influencing this; the underlying generating language model may simply be predicting common words with a significantly higher or lower probability than typically seen in human text. To test this, we took 500 tokens with the largest respective weights, and for each calculated the proportion of appearances in human examples against machine examples in the training set. We plotted each token's ratio against its weight, and got the graph shown in Figure 6. We can clearly see the positive trend between weight and ratio; as the proportion of occurrences in human examples gets greater than in machine examples, the weight also increases. This leads to the activation of our output neuron being larger, which makes it more likely that it will be classified as a human example.

4.3.2 Binary BoW. We also experimented with using a Binary Bag-of-Words approach. Given some text T and vocabulary V, for every word $m_i \in V$ we set $f_i = 1$ if m is present in T, if not we set $f_i = 0$.

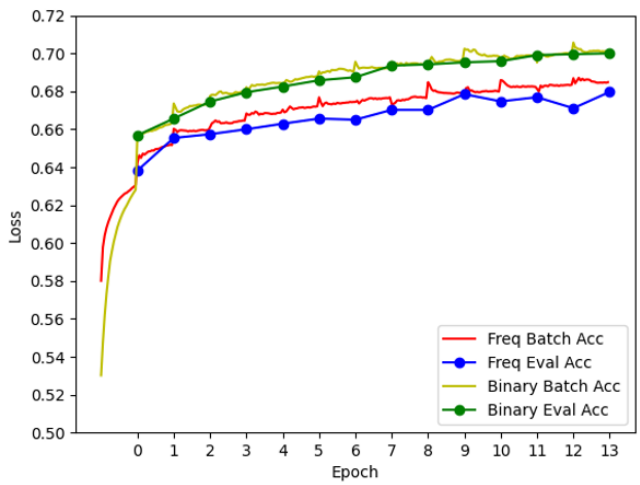

Figure 7: Accuracy over the training of binary BoW against frequency BoW

We took the concatenation of these to form our input vector $F = [f_1; ...; f_{|V|}]$. With this approach, we could find whether the actual frequency of tokens is significant for our classification task. We used the same architecture as our single-layer neural network and used the same dataset.

The accuracy over the training period is shown in Figure 7. Interestingly, we see that this methodology achieves a non-trivial higher accuracy, achieving 70.11%. This suggests that knowledge of which words are present, rather than the actual frequency counts, is more indicative of the text being the correct class. We also analysed the words belonging to the largest weights and saw that this model weighted more common words than our previous model, as shown in Figure 8.

4.3.3 Multi-layer Network. To improve the accuracy of this technique we also created a model using a multi-layer neural network. A single layer is very limited in its expressibility; it can only classify based on a simple weighted summation of the input vector. Adding more layers allows for interaction between the inputs, allowing the network to model more complex relationships. We added two hidden layers: the first with 512 neurons, and the second with 192 neurons. Both used ReLU [19] as non-linearities. Figure 9 displays the loss of our multi-layer neural network over training, with the loss of the single-layer neural network also plotted for comparison. We can see that our new architecture achieved a much lower loss, but likely overfitted during the last few epochs of training as the training error diverged from the evaluation error.

We also plotted the accuracy over training of our new technique against the old, shown in Figure 10. Our new model achieved an accuracy of 70.05% before it began to overfit. This new architecture was likely overfitted as it is more complex compared with the original model, thus it had the potential to quickly memorise the training set.

Token	Weight
including	0.267
In	0.235
The	0.211
also	0.210
<whitespace>The	0.208

Figure 8: Largest and smallest weights, with respective tokens, of the single-layer binary neural network

4.4 Total Probability

For our generators and discriminators we took the family of pre-trained GPT-2 models. This allowed

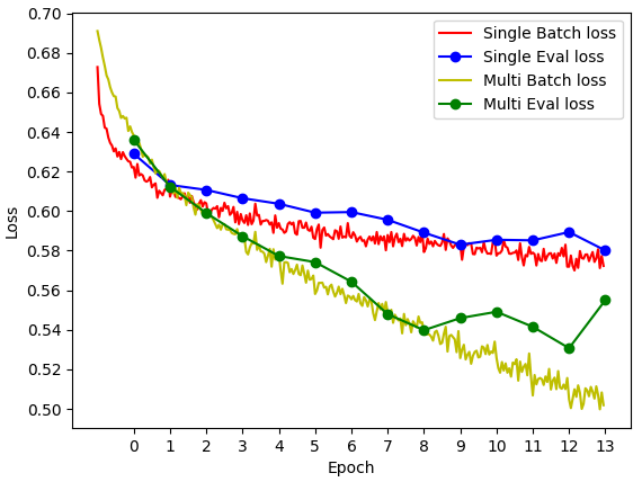

Figure 9: Loss over time for single- and multi-layer neural network

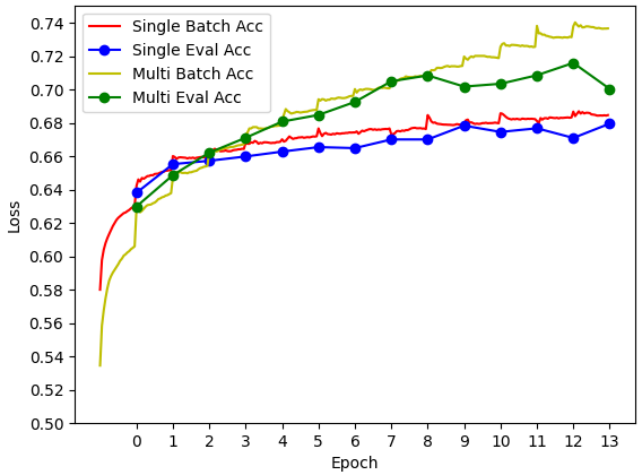

Figure 10: Accuracy over time for single- and multi-layer neural network

us to test the robustness of this method against model size, which is critical in the current climate where model size is rapidly increasing every year [24]. We used the WebText dataset to finetune each of the generators to predict news articles.

For a specific generator G and discriminator D, we took 5000 examples from the human dataset and generated 5000 examples using generator G. We then calculated the probability of D producing each example using equation 18. All probabilities were normalised by the respective text length, as not all examples were the same length. Using such data we used a decision stump to find a threshold that gave the best classification accuracy.

We expect the probability of long texts produced by GPT-2 to be different to that of human-written text due to a phenomenon known as *exposure bias* [36]. Language models are trained to maximise the equation shown below:

$$p(x) = \prod_{i}^{n} p(x_i | x_1, ..., x_{i-1}) \tag{19}$$

However, LLMs are trained only on human examples, not on its generations. Thus, during inference, the accuracy of the model will be hindered as it is not trained to produce text conditioned on its previous text. As a result, the perplexity of GPT-2 text will significantly differ over time from that of human text. This disparity in perplexity allows this method of thresholding the probability to be accurate, but only for long texts.

4.4.1 Experiments. We first tested this with GPT-2 117M as both the generator and discriminator, resulting in an accuracy of 76.50%. To help visualise the thresholding and offer limited interpretability to this process, we fit a distribution of probabilities as a Gaussian distribution and plotted them, as seen in Figure 11. What we see is that the human text has a higher mean than machine text, with a smaller standard deviation. We hypothesise that it has a higher mean due to the previously discussed exposure bias problem; models assign a

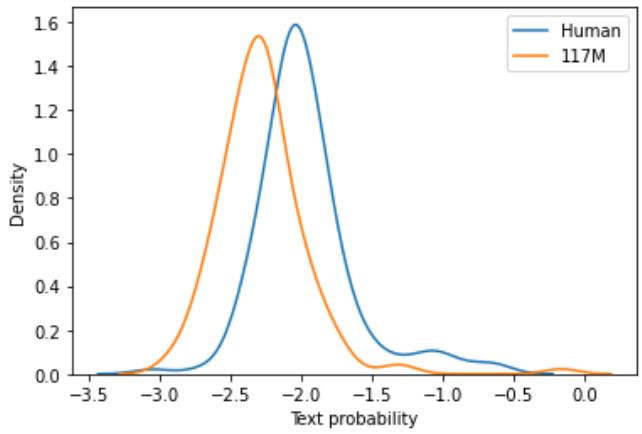

Figure 11: GPT-2 117M generator with GPT-2 117M discriminator

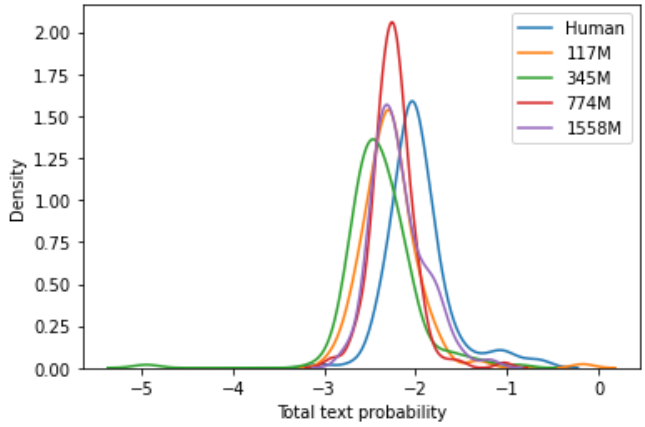

Figure 12: GPT-2 117M discriminator with all generators

higher probability to their training dataset rather than their generations.

To test how scaling the generator changes the resulting distribution, we paired each GPT-2 model as a generator against GPT-2 117M as the discriminator, and plotted the distributions in Figure 12. As the generator size increases, we observe that the standard deviation decreases; this is likely because these LLMs produce more coherent outputs as they become more capable.

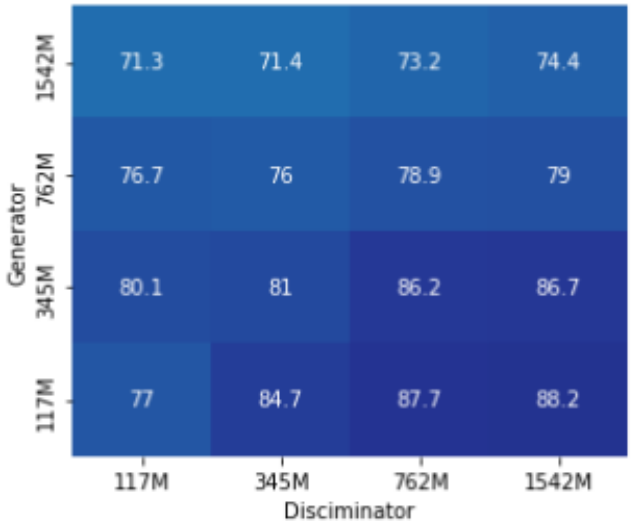

Figure 13: Accuracy of Total Probability method across all generator+discriminator combinations

4.4.2 Robustness. To further test the robustness of our methodology to see how our approach scales, we repeated this with every combination of the family of GPT-2 models as generators and discriminators and recorded the accuracy of the fit threshold. Our results are shown in Figure 13. Note, we performed these experiments with the vanilla model configuration, without including Top-p or Top-K. These configurations leave many artefacts within the output, thus our results are an underestimate.

Discriminators greatly improve in accuracy as their size increases with respect to the generators, across all size generators. However, as generators increase in size the accuracy rapidly falls, to as low as 71.30% when pairing GPT-2 117M discriminator against GPT-2 1524M generator. This reveals the underlying problem with this approach; it fails to

scale against more modern language models. Modern LLM parameters have recently been increasing at an exponential rate [24], thus this approach would continuously need to be re-trained on the newest publiclly accessible model.

4.5 Transformer

For our final classifier, we followed the methodology described in Section 3.4.6. Due to hardware constraints, we used GPT-2 117M, which is significantly less computationally expensive than the largest GPT-2 model. We removed the language-modelling head and instead used a single-layer network. We added a special token to the vocabulary and placed it at the end of every example. The final hidden state was extracted at this point and fed into the linear layer to classify the text.

This methodology achieved an accuracy of 91.03%. When tested on the larger GPT-2 models, 345M, 774M and 1558M, we got accuracies of 90.84%, 87.51% and 86.25% respectively. Zellers et al. [36] offers insight as how models can accurately identify outputs from other models. The authors suggest that specific sampling techniques leave artefacts in the output texts as they prevent the "tail" of the probability distribution of tokens from being sampled. Some discriminators can accurately pinpoint this tail from generators, and thus can accurately identify the provenance by detecting whether this tail is present in the given text. The authors suggest that the discriminator best pinpoints this tail when it models language in a similar fashion to the generator, thus this approach works best when considering models with similar architectures, training processes and configurations. This is supported by the arguments made in Uchendu et al. [29].

Interestingly, Solaiman et al. [27] claims that bi-directional discriminators outperform uni-directional discriminators for this task. Perhaps this is due to bidirectionality having the ability to capture information from both sides of a token. This is usually undesired for the task of language modelling, since it is often redundant and can confuse the model, but in discrimination, it would allow for more context to be factored into the classification. The authors also found this method proved to generalise well across different domains when using a more generalised dataset containing a large domain of topics with random-length sequences of text.

Despite achieving the best accuracy, this model has the worst interpretability of all our classifiers', thus we are unable to offer in-depth insight on how exactly this high accuracy is achieved. The transformer-decoder base is a gigantic black box that builds an accurate representation of the input. Research on transformer interpretability has been performed, but often this is for very specific tasks [6]; we couldn't find any research for our task.

5 ADVERSARIAL ATTACKS

Research in detecting language model outputs has led to an arms race; significant research has been conducted in mitigating the effectiveness of classifiers [27]. Simple techniques, such as altering individual characters or words, have led to certain classifiers having worse than random accuracy [16]. To further test the robustness of our techniques,

we evaluate our models when these attacks are employed and give explanations as to why the results changed.

5.1 Techniques

In this section, we follow the techniques conducted in Wolff and Wolff [35] to limit the effectiveness of classifiers by applying simple attacks.

5.1.1 Homoglyphs. The first class of attacks they introduced were non "human-like" attacks, which altered the text in ways that humans typically wouldn't. They researched swapping individual characters with their Unicode homoglyphs (e.g. swapping the English "A" to Greek "A"). This results in a visually identical text that has a different representation when tokenized and converted into word embeddings.

Most language models use a tokenization process known as Byte Pair Encoding (BPE) [10], which constructs tokens from other tokens, with priority given to sub-words that appear more frequently in the corpus. The dataset used for training GPT-2 contained predominantly English text because they removed non-English webpages [23]. As a result, tokens containing non-English characters are rarely created. During training, the word embeddings corresponding to such tokens are not updated significantly due to their infrequency in the dataset. Thus, when these word embeddings are introduced, they are effectively untrained word embeddings which give unsatisfactory results for the classification task.

We used a dataset that 1861 homoglyphs for characters in many different languages, from Russian to Mandarin [11]. For our test set from the Webtext dataset, we randomly replaced 1.5% of all characters in each example.

5.1.2 Misspelling Attack. The other class of attacks they discussed were "human-like" attacks, which result in visually different texts. These can be replacing words with their misspellings, or changing words to introduce semantic or syntactic errors. This methodology can result in a massive decrease in the accuracy of some language models, simply due to how they're trained. Many language models are trained predominantly on articles, academic papers and Wikipedia [5] [23] [20]. These are all well-composed texts that likely contain very few mistakes, thus the language models won't be exposed to these mistakes often. As a result, they don't learn a significant relationship between words and their misspelt variants, thus slight misspellings have a significant impact on the resulting classification.

We constructed a dataset using a list from Wikipedia of common misspellings, where for each misspelling they included the correct spelling [12]. For each example in our test dataset, we changed 5% of words with their respective misspellings.

5.2 Results

In this section we analyse the results of applying homoglyph attacks to our classifiers' and offer insight into why the accuracy changed.

The results of using the homoglyph and misspelling attacks are shown in Figures 14 and 15, with the multiplicative changes in accuracy given.

[11] https://github.com/codebox/homoglyph

[12] https://en.wikipedia.org/wiki/Wikipedia:Lists_of_common_misspellings/For_machines

Model	Δ Accuracy
Decision Tree	-10.25%
Random Forest	-17.45%
Linear BoW	-32.13%
Multi-layer BoW	-28.51%
Total Probability	-6.32%
Transformer	-18.49%

Figure 14: Homoglyph attack on classifiers

Model	Δ Accuracy
Decision Tree	-5.27%
Random Forest	-19.12%
Linear BoW	-29.55%
Multi-layer BoW	-27.82%
Total Probability	-12.42%
Transformer	-19.13%

Figure 15: Misspelling attack on classifiers

We see that most models suffer a significant decrease in accuracy, with our simplest models being brought down to an accuracy close to random.

All our models suffered a decrease in accuracy as they weren't exposed to such a large frequency of homoglyphs or misspellings in their training. As such, embeddings corresponding to homoglyphs and misspellings are fairly orthogonal to the embeddings of the unaltered text. Furthermore, our classifiers' themselves aren't robust to such changes.

Interestingly, all classifiers predicted human-written more frequently when these attacks were applied. Wolff and Wolff [35] claim this is because these classifiers are performing a separate task to what we believe them to be doing; rather than distinguishing between human- and machine-written, the authors claim that such classifiers instead identify what is and isn't indicative of a text being machine-written. Thus, introducing such attacks will vastly increase the probability of predicting the human tag.

Increasing the variety of styles and domains within our datasets would mitigate these attacks, as our trained models would be significantly exposed to these examples. Additionally, they'll build a relationship between homoglyphs in their word embeddings which will further increase the robustness.

6 INTERFACE

We required an interface to allow users to access our developed classifiers. This interface will take some text as input, run it through all the necessary classifiers and produce a final output on the likelihood of the input being human text.

6.1 Architecture

We decided to implement such an interface through a browser extension. This would fulfil our requirements for accessibility since they can be readily accessed anywhere on the internet. Additionally, installing and using such an application would be a very simple process compared to having to download and install a regular stand-alone program.

Our browser extension was implemented in Google Chrome. We recognise that not all use Google Chrome, but it is the most popular browser [13] and developing a similar web extension for other browsers would be a simple process.

[13] https://gs.statcounter.com/browser-market-share

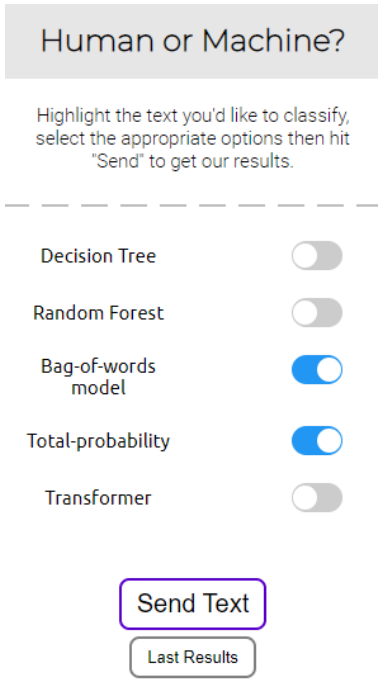

Figure 16: Initial view of our application

We first implemented the frontend of our application. Our initial view is shown in Figure 16. We included a brief, intuitive description of how the user should interact with this application. We next provided a list of toggleable options for each classifier to allow the user to select which ones they would like applied to the input highlighted text. The selection of these options is saved in Chrome's storage, so the state is preserved when re-opening the application. Once done the user would select the "Send Text" button. To prevent errors, we ensured this button could only be pressed when some text was highlighted and at least one classifier was selected. If either of these wasn't the case and the user attempted to select the button, the application would notify the user and explain what was wrong. We also included an option for the user to view the results of the last classification performed.

Once all checks had been passed and the button was pressed, an HTTP POST request was made to our backend server. The body of this request sent the selected text and the states of all options, all contained in a JSON string. This backend server was hosted in Django [14]. We chose Django due to its simplicity and previous developer experience using it. Furthermore, Django is a Python library, which is the same language we used to create the classifiers, thus integration was seamless. Hosting all models on a single device had its advantages; all models could be loaded at once, thus inference on any would be very quick. In comparison, if the user had to host the models themselves, even with a powerful computer inference would be slower due to having to load the model in/out of memory.

Given a request, the backend first parsed the body. The input text was converted to UTF-8 and cleaned. We then passed the cleaned text into the correct classifiers, depending on what options were selected by the user. All outputs were recorded along with information used for classifier interpretability. Next, we calculated the final classification output by using an ensemble of the selected models. We experimented with many different methods and settled on taking a weighted average of the classifier outputs. All information was stored in a JSON string and given to the frontend through an HTTP response.

Whilst the backend processes the input, the user is shown a loading view with text to signal that they need to wait. This is shown in Figure 17.

When the frontend receives the HTTP response, it parses the given results and stores them in Chrome

[14] https://djangoproject.com

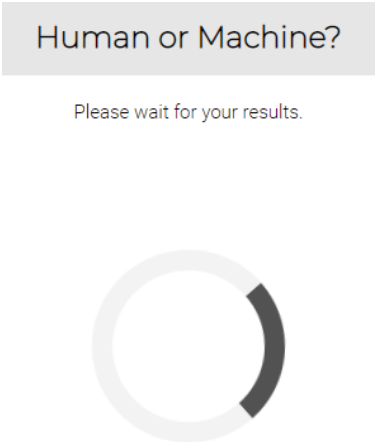

Figure 17: Loading view of the application

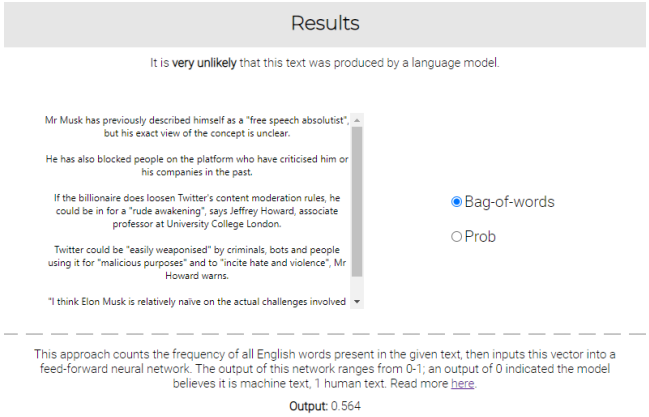

Figure 18: Results view of the application, with BoW model selected

storage, so they can be viewed later. We then inject all relevant information into the view. An example of this view is shown in Figure 18. Here, we took an extract from a BBC news article as input [15]. At the top we show the final result of our classification. We output this as a discrete set of sentences depending on the underlying probability: "It is [very unlikely/unlikely/likely/very likely] that this text was produced by a language model". The original text is displayed on the left; note how the format of the text is preserved. If the text is too long then it will be contained in a box with a scroll bar. On the right is a list of the selected classifiers. At the bottom is a high-level description of the selected classifier, with the raw output displayed alongside. If the total probability model is selected then the threshold will also be displayed.

[15]https://www.bbc.co.uk/news/business-61226282

Depending on the selected classifier we also offer limited interpretability to the user. For the total probability method, we visualise the probability of every single token within the text given all previous tokens. An example of this is shown in Figure 19, using the same BBC article as our text. Since most of the tokens are highlighted in red, most of the tokens had a very low probability of being produced by GPT-2. This visualisation can also help detect hybrid texts containing both human- and machine-written text; we can see how the probability of tokens changes over the course of the text.

7 EVALUATION

In this section we discuss the process of evaluating our solution and perform a critical review of our work in this project. We finish by covering the contributions of this project and reflecting on our project management skills.

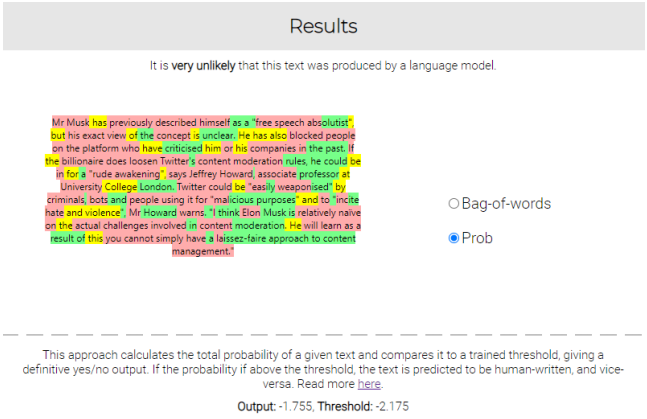

Figure 19: Results view of the application, with Total Probability model selected

Model	Accuracy
Decision Tree	57.35%
Random Forest	64.40%
Linear BoW	67.97%
Multi-layer BoW	70.05%
Total Probability	88.20%
Transformer	91.03%

Figure 20: Accuracies of implemented classifiers

7.1 Solution

Our focus for this project was primarily on the classifiers since the usefulness of the whole application depends on how good they are. No matter how great our interface looks and performs, if the classifiers themselves have low accuracy then the whole application is useless.

Throughout the process, we continuously tested the accuracy of our classifiers', and where possible we attempted to discern why specific classifiers achieved the accuracy they did. We implemented five classifiers, achieving 91.03% as our best accuracy. A full list of our accuracies for each classifier is shown in Figure 20. Of course, these are only relevant when dealing with the specific language models and configurations we worked with; we will discuss this further in Section 7.4.

All our classifiers' are inexpensive to run and scale. Our BoW model is a simple frequency count over some text that is fed into a feedforward neural network. The frequency count is $O(n)$ of the input length of tokens n, and the feedforward neural network has a constant time complexity irrespective of the length of the original text. Our Total Probability and Transformer methods both relied upon fairly large transformers. Their time complexity scales quadratically with respect to the length of the input [30], however, with our models the length of the input is capped at 1024 tokens [23].

To test robustness, for some models we calculated how their accuracy changed as we varied the architecture and model configuration they were paired with. However, we did identify the severity of adversarial attacks.

Unfortunately, we were only able to perform significant interpretability on a single of our classifiers'. All other models were too large or complex to analyse. We believe this is a significant hindrance to our solution, as it is impossible to understand why we get specific results.

7.2 Interface

We implemented an accessible interface that any user using Google Chrome could use. Our models were hosted on our hardware, eliminating the

technical and computational requirements. Furthermore, we designed our interface to integrate well with Google Chrome; users could simply highlight the text they desired to classify, toggle options to select the classifiers and click a button.

We gave a brief description of how each of our classifiers' worked and gave a link to where users could read more in-depth about how that specific classifier works. This paper and our full application will be made publiclly accessible, so users can see the exact implementation details. We were also able to display interpretability for our Bag-of-Words method.

To test the robustness of our interface, we checked all edge-cases of inputs didn't result in any errors. We inputted many different formats of text to check our cleaning pipeline would correctly format the input for our classifiers'. Furthermore, we checked that all likely classifier errors were translated into a user-friendly message, so technically-inexperienced users would understand how to solve the problem.

7.3 Appraisal

We found that our progress was often constrained by our inexperience in natural language processing. Since this classification problem employs cutting-edge artificial intelligence research, a broad literature review was performed early in the process to understand the field better. Unfortunately, this wasn't enough, and we found we had to continuously spend significant effort educating ourselves to better understand the underlying technologies. Due to this inexperience, a huge amount of effort was expended in solving problems that weren't necessary for this project. For example, we spent a few weeks creating a huge dataset through web scraping and training a language model ourselves, however, we realised that we had vastly overestimated the required size of the dataset. Another example is when we entirely changed the direction of our project. We originally wanted to build a tool that would verify some arbitrary claim on the internet by finding articles on the same topic, but after reviewing existing methods we realised that it would be infeasible to make a significant contribution. In response we pivoted our project to just look at bot-produced texts. A better-planned process would have alleviated this, as we would have foreseen more problems.

This project improved our ability to conduct independent research effectively. We learnt how to perform an effective literature review, linking concepts and arguments between many different papers, and deduce what areas of the field require additional work. We also gained knowledge in practising machine knowledge for research.

7.4 Limitations

Unfortunately, our project also has its limitations. Since we were constrained by hardware requirements, we often used the smallest publiclly-accessible GPT-2 model despite the largest language model containing almost 5000 times as many parameters [8]. The architecture of many modern language models has drastically changed, as the trend of simply scaling up transformers has (luckily) begun to stop. As explained earlier, even the best modern classifiers aren't very robust and often perform

badly when tested on larger models. Thus, our application may prove ineffective when applied to some outputs.

Additionally, our lack of interpretability also hinders our application. Our classifiers' will likely mirror biases from the GPT-2 models; aside from our simplest BoW Linear model, we can't decipher why exactly this is.

8 CONCLUSION

The project was a success. We developed several accurate classifiers and integrated them into an accessible application to allow individuals and groups to fight misinformation, propaganda and many other malicious applications.

We met all the objectives that we stated in Section 2. We produced several accurate classifiers, all of which were scalable. We also tested robustness and demonstrated that some performed well against larger models; our Transformer model could detect the outputs of a model over 10 times its size with an accuracy of 86.25%. These were all integrated within an interface, that was simple and accessible enough such that most could use the application without any technical machine learning experience.

We could have made our models more robust against adversarial attacks. Through testing we showed that these attacks could severely harm the accuracy of our models; in some cases, it brought them close to random accuracy. Additionally, further tests on the robustness of the latest generation of language models could have been performed. Finally, we should have trained our classifiers' such that they are more likely to produce false negatives rather than false positives, even if this did come at the cost of lower accuracy.

8.1 Future Work

After reflecting on our project and surveying the state of the field, we identified several areas that would benefit from further research.

8.1.1 Watermarking. Different solutions for classifying text should be explored further. Our methodology of calculating the likelihood of a raw output is surprisingly effective, yet may not be robust as the field of natural language processing changes. Abdelnabi and Fritz [1] proposes a fruitful methodology, where they augmented a language model to include a watermarking process in its pipeline, such that outputs are "watermarked" with a specific number.This watermarking process shouldn't alter the meaning of the text or introduce any semantic or syntactic errors. Additionally, a decoder should be trained that can identify the watermark of a given text. By giving each language model a unique watermarking number, we can then identify outputs by passing them through the decoder, and additionally even track the exact generating language model. This approach is somewhat similar to Tay et al. [28] which we discussed in Section 3.4.5, but here we deliberately inject specific watermarks within the text to be detected.

8.1.2 Interpretability. Further work on creating interpretable classifiers is crucial for this task. Better understanding how our classifiers' work allows us to alter them so they align with our interests more. For example, we could remove biases within our systems easier. Unfortunately, only one of my

classifiers had any interpretability at all, and it was one of my least accurate classifiers. The current state-of-the-art classifier is to use the largest transformer-based language model, and these models containing billions of parameters are difficult to understand. Further research in transformer interpretability, or research in simpler classification methods, will further this cause.

8.1.3 Auxiliary Signals. One potential improvement to our classifiers' would be to leverage auxiliary signals. Incorporating external world knowledge into our classification would improve accuracy; our models could verify statements and judge their likelihood. This is achievable for a specific set of simple auxiliary signals, for example, we could manually give models access to sports results when classifying sports articles. However, it is unknown how to create a generalised solution to this. Perhaps simply scaling up transformer-based language models will lead to a greater ability to recognise such signals, leading to higher accuracy.

8.1.4 Robustness. Finally, more research into robust classifiers is crucial. A fault with all classifiers is that they fail to scale as the size of the LLM increases, thus all trained classifiers will be redundant within a few years of creation. This proves to be a problem when training such classifiers is an expensive and time-consuming task.

8.2 Legal, Ethical and Social Issues

Our solution has several problems that should be addressed. We aim to make all users of our application aware of these through our interface.

Two of our implemented classifiers rely on LLMs, specifically GPT-2. Unfortunately, GPT-2 has inherent biases and assumptions encoded within its weights, which can lead to toxic texts being produced when GPT-2 is applied to the task of language modelling [27]. GPT-2 has been shown to exhibit gender bias, racial bias, religious bias, language preference and more. This bias exists because the dataset it was trained on contains such qualities. GPT-2's loss function means it will try and mimic its training dataset as best as possible, thus it will mimic prevalent biases. In our case, classifiers using GPT-2 will change their output depending on the mentioned qualities above. Thus, if our application were to be deployed to some social media website, it would disproportionately label specific groups with the incorrect label. Users of this application must be adequately informed about these problems so they can make their judgement in light of this.

Secondly, we must ensure that our results are not treated as a source of ground truth. In addition to our transparency, we must explicitly state the flaws of our classifiers'. If we don't, people may overestimate the capabilities of the application.

8.3 Project Management

Since this was a research-oriented project, where the direction of progress may significantly change, we decided to adopt an agile methodology. The field of language models is moving rapidly, so we could quickly adapt to new research; for example, halfway through my dissertation research on a new LLM that was the new state-of-the-art was released [24].

We began by creating a Kanban board. Using this we could create incremental goals for each objective, and easily track progress on each objective. Since we performed an extensive literature review at the beginning of this project, we used Notion [16] to write summaries of papers and link concepts together. This proved especially useful when evaluating academic opinions on topics where no consensus had been reached. We started a work diary, which allowed me to track progress and thoughts over the course of the project. All supervisor meeting notes were also recorded within Notion.

Once development began, we extended by Kanban board by including a Gantt chart, so every objective would have a corresponding schedule. At the start of every term, we created a rough high-level plan of workflow to be carried out over the term whilst allowing myself the flexibility to adjust if necessary.

[16] https://www.notion.so/

REFERENCES

[1] Sahar Abdelnabi and Mario Fritz. Adversarial watermarking transformer: Towards tracing text provenance with data hiding. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 121–140. IEEE, 2021.

[2] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[3] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745*, 2022.

[4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. *arXiv preprint arXiv:2112.04426*, 2021.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[6] Hila Chefer, Shir Gur, and Lior Wolf. Transformer interpretability beyond attention visualization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 782–791, 2021.

[7] Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.

[8] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

[9] Joshua Fox and Carl Shulman. Superintelligence does not imply benevolence. *ECAP*, 10:456–462, 2010.

[10] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[11] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International Conference on Machine Learning*, pages 1243–1252. PMLR, 2017.

[12] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M Rush. Gltr: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[14] Joe Hewett. A gpt-3-based automated victim for advanced fee fraud disruption. 2022.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[16] Daphne Ippolito, Daniel Duckworth, Chris Callison-Burch, and Douglas Eck. Automatic detection of generated text is easiest when humans are fooled. *arXiv preprint arXiv:1911.00650*, 2019.

[17] Ganesh Jawahar, Muhammad Abdul-Mageed, and Laks VS Lakshmanan. Automatic detection of machine generated text: A critical survey. *arXiv preprint arXiv:2011.01314*, 2020.

[18] Peter J Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. *arXiv preprint arXiv:1801.10198*, 2018.

[19] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Icml*, 2010.

[20] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[21] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.

[22] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9, 2019.

[24] Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.

[25] Marc'Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732*, 2015.

[26] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, California Univ San Diego La Jolla Inst for Cognitive Science, 1985.

[27] Irene Solaiman, Miles Brundage, Jack Clark, Amanda Askell, Ariel Herbert-Voss, Jeff Wu, Alec Radford, Gretchen Krueger, Jong Wook Kim, Sarah Kreps, et al. Release strategies and the social impacts of language models. *arXiv preprint arXiv:1908.09203*, 2019.

[28] Yi Tay, Dara Bahri, Che Zheng, Clifford Brunk, Donald Metzler, and Andrew Tomkins. Reverse engineering configurations of neural text generation models. *arXiv preprint arXiv:2004.06201*, 2020.

[29] Adaku Uchendu, Thai Le, Kai Shu, and Dongwon Lee. Authorship attribution for neural text generation. In *Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[31] Ben Wang. Mesh-Transformer-JAX: Model-Parallel Implementation of Transformer Language Model with JAX. https://github.com/kingoflolz/mesh-transformer-jax, May 2021.

[32] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atoosa Kasirzadeh, et al. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*, 2021.

[33] Johannes Welbl, Amelia Glaese, Jonathan Uesato, Sumanth Dathathri, John Mellor, Lisa Anne Hendricks, Kirsty Anderson, Pushmeet Kohli, Ben Coppin, and Po-Sen Huang. Challenges in detoxifying language models. *arXiv preprint arXiv:2109.07445*, 2021.

[34] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/2020.emnlp-demos.6.

[35] Max Wolff and Stuart Wolff. Attacking neural text detectors. *arXiv preprint arXiv:2002.11768*, 2020.

[36] Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. Defending against neural fake news. *Advances in neural information processing systems*, 32, 2019.